

Side Information

- Repo: <https://github.com/ReitererLukas/constraint-coloring-in-c>
- Also submodule in: <https://github.com/kritika99k/topology-and-coloring>
- Written in cpp
- use `make test` to start verification
- see code at `tests/test_runner.cpp` -> `verify8colorsDegree4` (there is also `verify9colorsDegree4`, but that does not work)
- everything is 0 based. So if you have 9 colors the available colors are 0 to 8
- randomly searching for valid outputs is done in `main.cpp` (start with `make run arg`)

Things to write about

- Arrays and the index system
- `givenRand` arrays
- `generateOutput`
- `checkAllConstraints`

Definitions

- View = 1 hop neighborhood
- Center = central node in a view (has degree many edges)
- Neighbor = adjacent node to a central node in a view (has 1 edge)
- `numOfColors` = number of original Colors
- `numReductionColors` = number of colors we want to reduce to

How does the verification work

To ensure maximum efficiency, all data is stored on the heap and memory allocations are reused, so that no data has to be copied around and to reduce the number of memory allocations. The Implementation uses three arrays during its runtime. Each array has a fixed, precomputed length and compresses multidimensional lists or hash maps into one dimensional arrays. The combinations array stores all possible neighbor combinations for a specific center color. The i^{th} combination can be accessed by using the indexes $\{i * degree + 0, \dots, i * degree + (degree - 1)\}$. The output array stores pointers to each generated output. One output belongs to one node-edge pair (`nodeid`, (`edgeid1`, `edgeid2`)). To map a node-edge pair to an index the implementation uses the `NodeEdgeKey.toIndex(int degree)` method, which creates a 1:1 mapping.

If there is an edge between two colors x and y , all outputs based on node-edge pairs of these colors will be the same. And since $A = \{0, 1, \dots, degree\}$ is fixed and outputs where a node with color $a \in A$ are already defined, the algorithm is just interested in finding some output for the $edges \in B \times B$, with $B = \{degree + 1, numOfColors\}$ and therefore just interested in checking the constraints for views with $b \rightarrow b$ edges. Therefore the `givenRand` array stores the output for each $b \rightarrow b$ edge. Similiar to the outputs array the verification algorithm uses `EdgeKey.toIndex(int numOfColors, int numColorsReduction)` to map a $b \rightarrow b$ edge to the corresponding output. For verification purposes the

givenRand array is given by the user as input. But for finding a new valid solution the implementation uses `randomizeCollections` to fill the givenRand array with randomly generated collections.

The generateOutput function fills the output array with values. To reduce the number of unnecessary comparisons, the function leaves out all views without a $b \rightarrow b$ edge, as those views and corresponding outputs already satisfy the constraints by definition. As all $b \rightarrow b$ outputs are already defined in the givenRand array, the generateOutput function just has to point the output of the corresponding $b \rightarrow b$ mapping to the collection in the givenRand array

Last the verification algorithm checks if both node and edge constraints are satisfied by this output. Therefore it first iterates over every view and checks if the node constraint does hold for the center node. If all node constraints do hold the implementation checks all the degree many edges of each view.